



Weather forecast in a Dart console app

```
this is the forecast for Sydney:
2026-05-21 06:00:00.000+1000: 10.9°C, 0.0mm, 20%
2026-05-21 07:00:00.000+1000: 11.1°C, 0.0mm, 16%
2026-05-21 08:00:00.000+1000: 12.4°C, 0.0mm, 29%
2026-05-21 09:00:00.000+1000: 14.9°C, 0.0mm, 39%
2026-05-21 10:00:00.000+1000: 16.3°C, 0.0mm, 23%
2026-05-21 11:00:00.000+1000: 17.2°C, 0.0mm, 33%
2026-05-21 12:00:00.000+1000: 17.3°C, 0.0mm, 41%
2026-05-21 13:00:00.000+1000: 17.0°C, 0.0mm, 54%
2026-05-21 14:00:00.000+1000: 17.0°C, 0.0mm, 100%
2026-05-21 15:00:00.000+1000: 16.9°C, 0.0mm, 78%
2026-05-21 16:00:00.000+1000: 16.9°C, 0.0mm, 32%
2026-05-21 17:00:00.000+1000: 16.2°C, 0.1mm, 44%
```

 Open-Meteo UI 

Weather in: Sydney ▾

Forecast for Thu 21-May 06:00 (local time)



temp: 10.9°C

clouds: 20%

wind: 14km/h ▾

 ↑ 06:44 ↓ 16:59

 ↑ 11:15 ↓ 21:29

Thu 06:00		10.9°C	 20%	 0%
Thu 07:00		11.1°C	 16%	 0%
Thu 08:00		12.4°C	 29%	 0%
Thu 09:00		14.9°C	 39%	 0%
Thu 10:00		16.3°C	 23%	 0%
Thu 11:00		17.2°C	 33%	 0%
Thu 12:00		17.3°C	 41%	 0%
Thu 13:00		17.0°C	 54%	 0%
Thu 14:00		17.0°C	 100%	 1%
Thu 15:00		16.9°C	 78%	 2%
Thu 16:00		16.9°C	 32%	 10%
Thu 17:00		16.2°C	 44%	 29% (0.1mm)

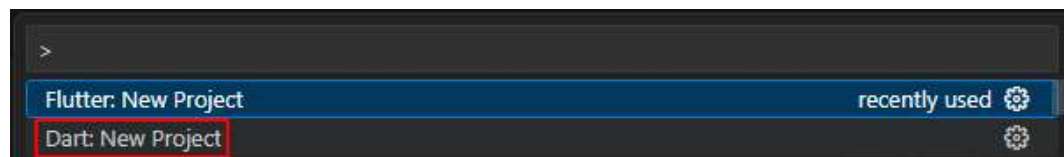


Create new Dart project in VS Code

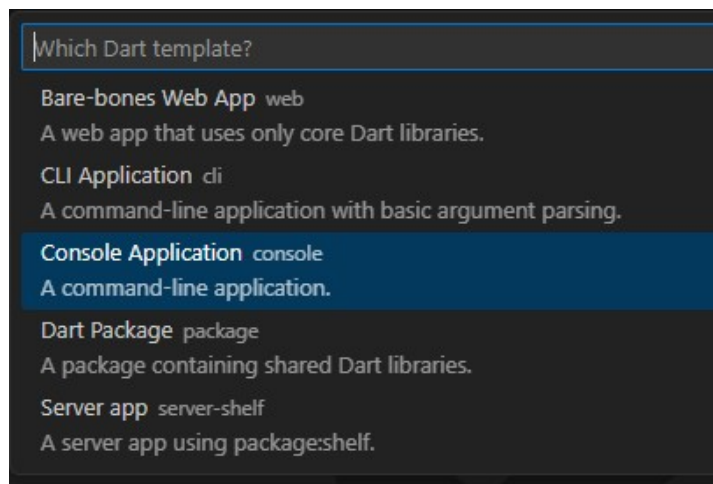
It's similar to creating a new Flutter project:

Open a new VS Code, in case it opens a folder, close this folder.

Then press F1 and select



In next step select:





Create new Dart project in VS Code (part2)

In next step select the folder, where the dart project should be created, e.g. C:\flutter\code and enter the name of the new app:

Should have been
weather_cli_2026

weather_cli_2016
Enter a name for your new project (Press 'Enter' to confirm or 'Escape' to cancel)

Now the project gets created:

 dart create -t console weather_cli_2016 --force: console... 
Source: Dart 

And finally it is opened in VS Code:



Create new Dart project in VS Code (part3)

The screenshot shows the VS Code interface with a Dart project named 'weather_cli_2016'. The Explorer panel on the left shows the project structure: 'WEATHER_CLI_2016' containing '.dart_tool', 'bin', 'lib', 'test', and various configuration files. The 'bin' folder is selected, and the 'weather_cli_2016.dart' file is open in the editor. The code in the editor shows the main function and a call to the 'calculate' function. The 'lib' folder is also selected, and the 'weather_cli_2016.dart' file is open, showing the 'calculate' function implementation.

```
bin > weather_cli_2016.dart > ...
1 import 'package:weather_cli_2016/weather_cli_2016.dart' as weather_cli_2016;
2
3 void main(List<String> arguments) {
4   print('Hello world: ${weather_cli_2016.calculate()}!');
5 }
6
```

```
lib > weather_cli_2016.dart > calculate
1 int calculate() {
2   return 6 * 7;
3 }
4
```

The “**bin**” folder contains the entry point of your app (the main function), while in the “**lib**” folder you define the classes that implement your app’s functionality. In the “**test**” you can define automatic tests of your app, which will be discussed later.



Alternative way to create a new Dart project

Open a new Terminal or Command Prompt in the directory, where your project should be created and enter the command: “dart create -t console weather_cli_2016”

```
PS D:\_tmp\xxx> dart create -t console weather_cli_2016
Creating weather_cli_2016 using template console...

.gitignore
analysis_options.yaml
CHANGELOG.md
pubspec.yaml
README.md
bin\weather_cli_2016.dart
lib\weather_cli_2016.dart
test\weather_cli_2016_test.dart

Running pub get...
Resolving dependencies...
Downloading packages...
Changed 48 dependencies!

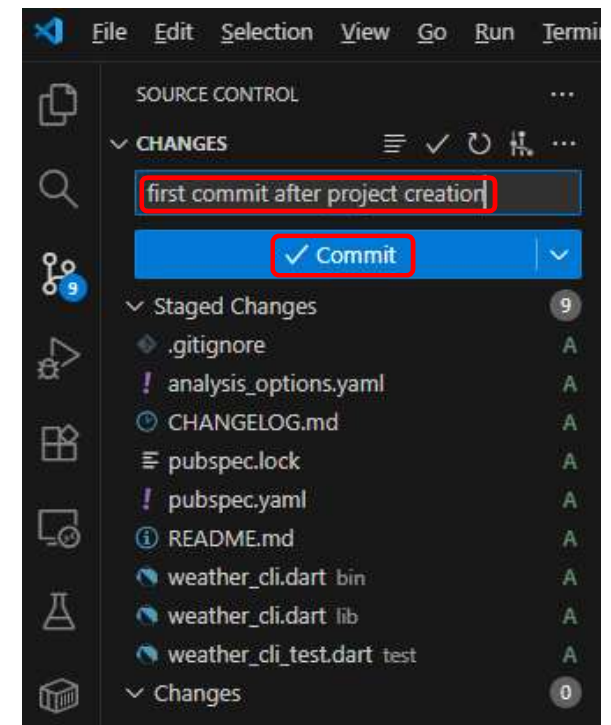
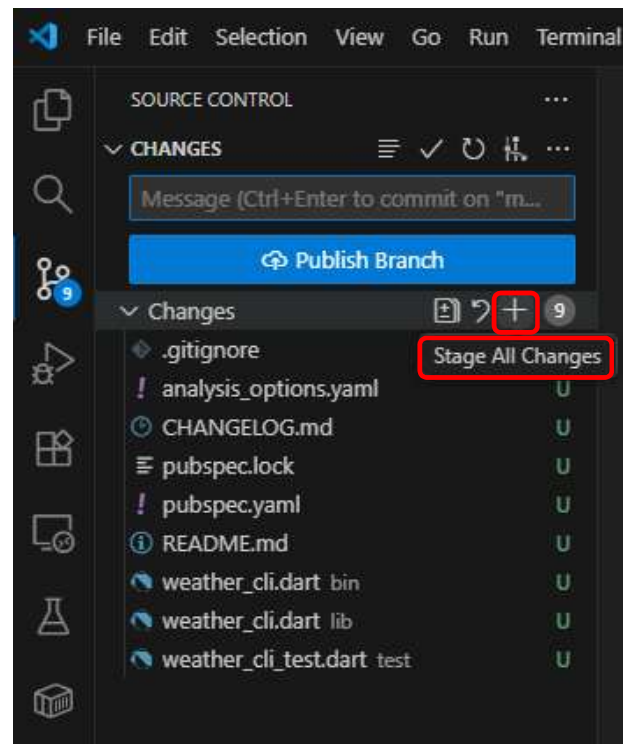
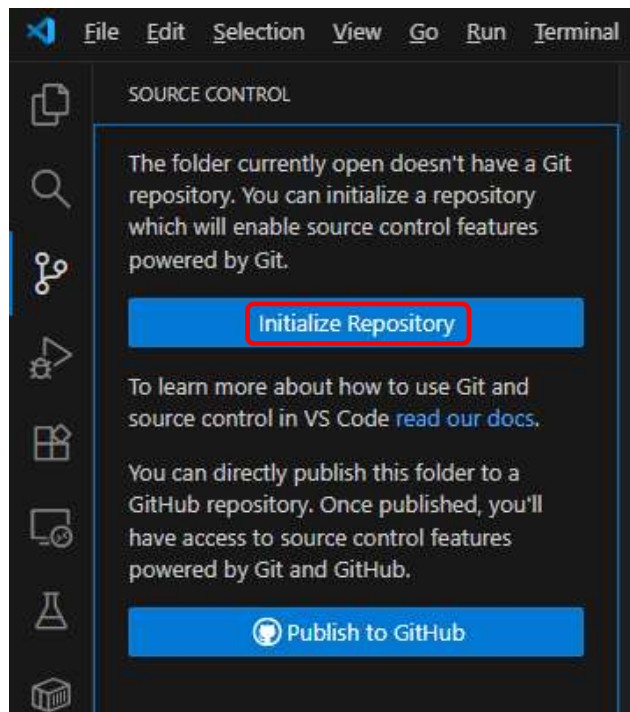
Created project weather_cli_2016 in weather_cli_2016! In order to get started, run the following commands:

cd weather_cli_2016
dart run
```

The created project structure is the same as the one created by VS Code.

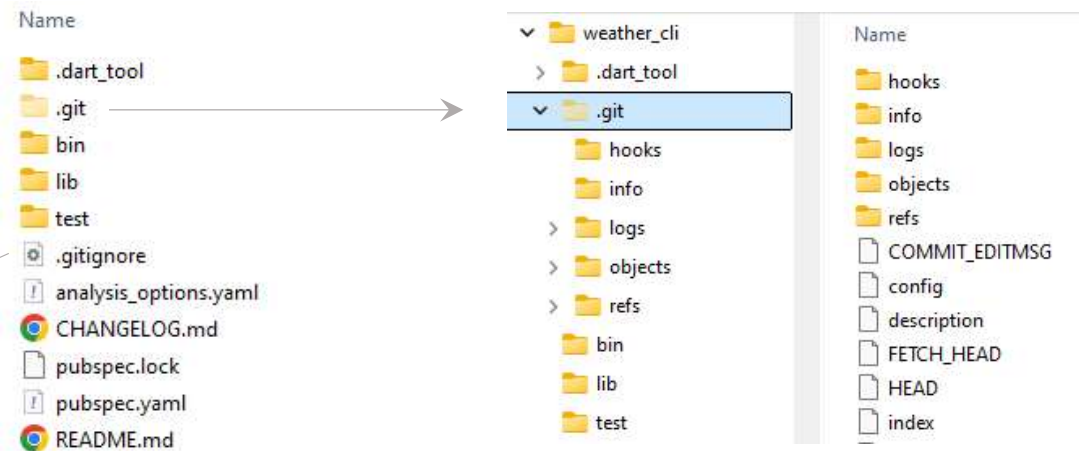


Initialize git repository





.git directory and .gitignore file

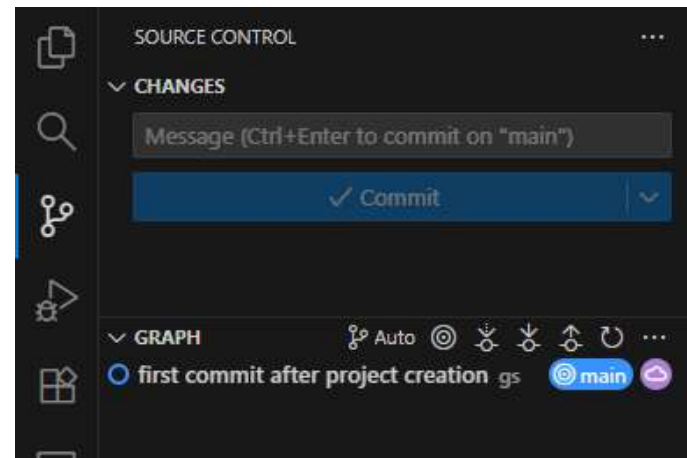
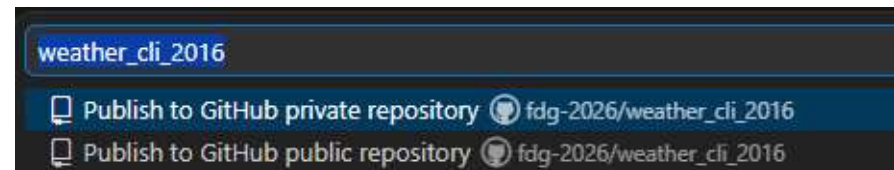
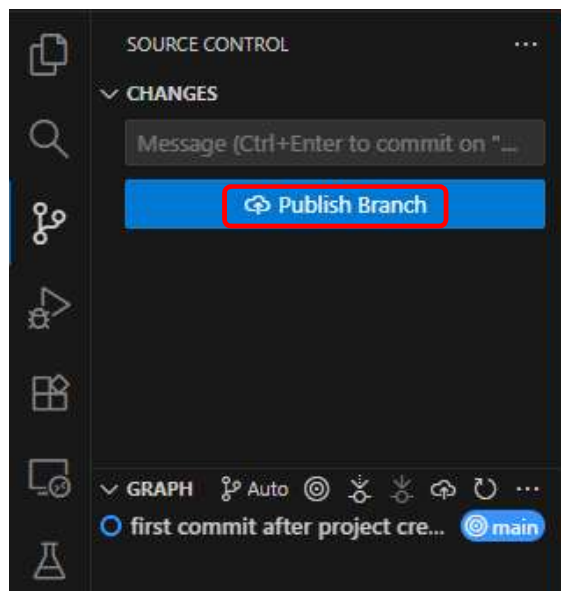


```
.gitignore x
# https://dart.dev/guides/libraries/private-files
# Created by `dart pub`
.dart_tool/
```

.gitignore describes which files stay out of the repository.

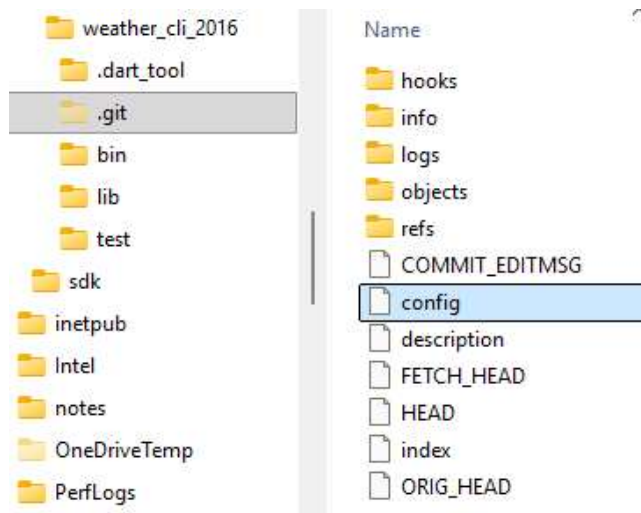
Folder .git contains the local repository.

Publish git repository





Forgot the URL of the remote repository ?



```
config x
[core]
  repositoryformatversion = 0
  filemode = false
  bare = false
  logallrefupdates = true
  ignorecase = true
[remote "origin"]
  url = https://github.com/fdg-2026/weather_cli_2016.git
  fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
  remote = origin
  merge = refs/heads/main
```



weather_cli_2016 Private Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags Go to file Add file Code

gs	first commit after project creation	72575e1 · 2 minutes ago	1 Commit
bin	first commit after project creation	2 minutes ago	
lib	first commit after project creation	2 minutes ago	
test	first commit after project creation	2 minutes ago	
.gitignore	first commit after project creation	2 minutes ago	
CHANGELOG.md	first commit after project creation	2 minutes ago	
README.md	first commit after project creation	2 minutes ago	
analysis_options.yaml	first commit after project creation	2 minutes ago	
pubspec.lock	first commit after project creation	2 minutes ago	
pubspec.yaml	first commit after project creation	2 minutes ago	

README

A sample command-line application with an entrypoint in `bin/`, library code in `lib/`, and example unit test in `test/`.

About

No description, website, or topics provided.

- Readme
- Activity
- 0 stars
- 0 watching
- 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors

No contributors

Languages

Dart 100.0%



Modify Readme.md

Created content:

```
README.md X
README.md
1 A sample command-line application with an entrypoint in `bin/`, library code
2 in `lib/`, and example unit test in `test/`.
3
```

Modified, pre-viewed,
committed, published ...

```
README.md •
README.md > abc ## WEATHER_CLI
1 ## WEATHER_CLI
2
3 This project was created during a Flutter
  training for students of [Friedrich Dessauer
  Gymnasium](https://fdg-ab.de/) in Aschaffenburg.
4
5 It is a Dart console application and uses [Open
  Meteo Api](https://open-meteo.com/) to retrieve
  hourly weather forecasts.
6
```

Preview README.md X

WEATHER_CLI

This project was created during a Flutter training for students of [Friedrich Dessauer Gymnasium](https://fdg-ab.de/) in Aschaffenburg.

It is a Dart console application and uses [Open Meteo Api](https://open-meteo.com/) to retrieve hourly weather forecasts.

New display on GitHub:

README

WEATHER_CLI

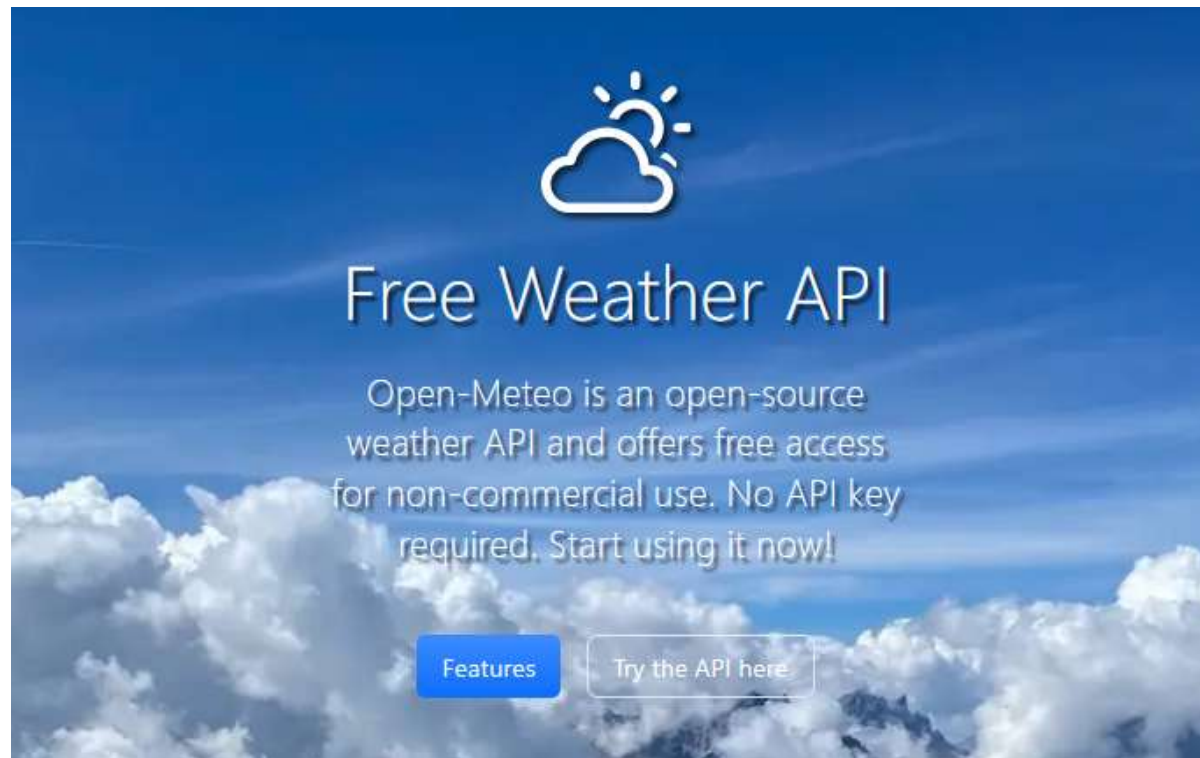
This project was created during a Flutter training for students of [Friedrich Dessauer Gymnasium](https://fdg-ab.de/) in Aschaffenburg.

It is a Dart console application and uses [Open Meteo Api](https://open-meteo.com/) to retrieve hourly weather forecasts.



API used to get weather forecast data

We will use <https://open-meteo.com/> :





Hourly forecast

Hourly Weather Variables

- | | | | |
|---|--|---|---|
| ☒ Temperature (2 m) | ☐ Weather code | ☐ Wind Speed (10 m) | ☐ Soil Temperature (0 cm) |
| ☐ Relative Humidity (2 m) | ☐ Sea Level Pressure | ☐ Wind Speed (80 m) | ☐ Soil Temperature (6 cm) |
| ☐ Dewpoint (2 m) | ☐ Surface Pressure | ☐ Wind Speed (120 m) | ☐ Soil Temperature (18 cm) |
| ☐ Apparent Temperature | ☒ Cloud Cover Total | ☐ Wind Speed (180 m) | ☐ Soil Temperature (54 cm) |
| ☐ Precipitation Probability | ☐ Cloud Cover Low | ☐ Wind Direction (10 m) | ☐ Soil Moisture (0-1 cm) |
| ☒ Precipitation (rain + showers + snow) | ☐ Cloud Cover Mid | ☐ Wind Direction (80 m) | ☐ Soil Moisture (1-3 cm) |
| ☐ Rain | ☐ Cloud Cover High | ☐ Wind Direction (120 m) | ☐ Soil Moisture (3-9 cm) |
| ☐ Showers | ☐ Visibility | ☐ Wind Direction (180 m) | ☐ Soil Moisture (9-27 cm) |
| ☐ Snowfall | ☐ Evapotranspiration | ☐ Wind Gusts (10 m) | ☐ Soil Moisture (27-81 cm) |
| ☐ Snow Depth | ☐ Reference Evapotranspiration (ET ₀) | ☐ Temperature (80 m) | |
| | ☐ Vapour Pressure Deficit | ☐ Temperature (120 m) | |
| | | ☐ Temperature (180 m) | |

Forecast days
1 day

Past days
0 days (default)

By default, we provide forecasts for 7 days, but you can access forecasts for up to 16 days. If you're interested in past weather data, you can use the **Past Days** feature to access archived forecasts.

API URL ([Open in new tab](#) or copy this URL into your application)

https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&hourly=temperature_2m,precipitation,cloud_cover&forecast_days=1

Berlin



API response in Chrome or Edge

https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&hourly=temperature_2m,precipitation,cloud_cover&forecast_days=1

The screenshot shows a web browser window with the URL `api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&hourly=temperature_2m,precipitation,cloud_cover&forecast_days=1`. The browser's developer tools are open, displaying the JSON response. A 'Pretty-print' checkbox is visible and checked. The JSON data includes metadata like latitude, longitude, and generation time, followed by an hourly forecast for temperature, precipitation, and cloud cover over a 24-hour period.

```
{
  "latitude": 52.52,
  "longitude": 13.419998,
  "generationtime_ms": 0.058650970458984375,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 38.0,
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "\u00b0C",
    "precipitation": "mm",
    "cloud_cover": "%"
  },
  "hourly": {
    "time": [
      "2025-11-10T00:00", "2025-11-10T01:00", "2025-11-10T02:00", "2025-11-10T03:00", "2025-11-10T04:00", "2025-11-10T05:00", "2025-11-10T06:00", "2025-11-10T07:00", "2025-11-10T08:00", "2025-11-10T09:00", "2025-11-10T10:00", "2025-11-10T11:00", "2025-11-10T12:00", "2025-11-10T13:00", "2025-11-10T14:00", "2025-11-10T15:00", "2025-11-10T16:00", "2025-11-10T17:00", "2025-11-10T18:00", "2025-11-10T19:00", "2025-11-10T20:00", "2025-11-10T21:00", "2025-11-10T22:00", "2025-11-10T23:00"
    ],
    "temperature_2m": [
      8.1, 8.2, 8.2, 7.8, 7.7, 7.8, 7.8, 7.8, 7.9, 8.2, 8.7, 9.4, 9.9, 10.3, 10.5, 10.2, 10.0, 9.7, 8.9, 8.0, 7.3, 6.7, 6.1, 4.6
    ],
    "precipitation": [
      0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.40, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00
    ],
    "cloud_cover": [
      100, 100, 98, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 99, 91, 100, 17, 28, 59, 12, 6, 100
    ]
  }
}
```

API response in Firefox



https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&hourly=temperature_2m,precipitation,cloud_cover&forecast_days=1

JSON

Raw Data

Headers

Save

Copy

Collapse All

Expand All

Filter JSON

latitude:

52.52

longitude:

13.419998

generationtime_ms:

0.061392784118652344

utc_offset_seconds:

0

timezone:

"GMT"

timezone_abbreviation:

"GMT"

elevation:

38.0 JS: 38

hourly_units:

time:

"iso8601"

temperature_2m:

"°C"

precipitation:

"mm"

cloud_cover:

"%"

hourly:

time:

0:

"2025-11-10T00:00"

1:

"2025-11-10T01:00"

2:

"2025-11-10T02:00"

3:

"2025-11-10T03:00"

4:

"2025-11-10T04:00"

5:

"2025-11-10T05:00"

6:

"2025-11-10T06:00"

7:

"2025-11-10T07:00"

8:

"2025-11-10T08:00"

9:

"2025-11-10T09:00"

10:

"2025-11-10T10:00"

11:

"2025-11-10T11:00"

12:

"2025-11-10T12:00"

temperature_2m:

0:

8.1

1:

8.2

2:

8.2

3:

7.8

4:

7.7

5:

7.8

6:

7.8

7:

7.8

8:

7.9

9:

8.2

10:

8.7

11:

9.4

12:

9.9

13:

10.3

14:

10.5

15:

10.2

16:

10.0 JS: 10

17:

9.7

18:

8.9

19:

8.0 JS: 8

20:

7.3

21:

6.7

22:

6.1

23:

4.6

precipitation:

0:

0.00 JS: 0

1:

0.00 JS: 0

2:

0.00 JS: 0

3:

0.00 JS: 0

4:

0.00 JS: 0

5:

0.00 JS: 0

6:

0.00 JS: 0

7:

0.00 JS: 0

8:

0.00 JS: 0

9:

0.40 JS: 0.4

10:

0.00 JS: 0

11:

0.00 JS: 0

12:

0.00 JS: 0

13:

0.00 JS: 0

14:

0.00 JS: 0

15:

0.00 JS: 0

16:

0.00 JS: 0

17:

0.00 JS: 0

18:

0.00 JS: 0

19:

0.00 JS: 0

20:

0.00 JS: 0

21:

0.00 JS: 0

22:

0.00 JS: 0

23:

0.00 JS: 0

cloud_cover:

0:

100

1:

100

2:

98

3:

100

4:

100

5:

100

6:

100

7:

100

8:

100

9:

100

10:

100

11:

100

12:

100

13:

100

14:

100

15:

99

16:

91

17:

100

18:

17

19:

28

20:

59

21:

12

22:

6

23:

100



Adding package http

Command is: `dart pub add http`

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\flutter\code\weather_cli_2016> dart pub add http
  Resolving dependencies...
  Downloading packages...
  + http 1.6.0
  Changed 1 dependency!
○ PS C:\flutter\code\weather_cli_2016> |
```

```
! pubspec.yaml M X
! pubspec.yaml
1  name: weather_cli_2016
2  description: A sample command-line application.
3  version: 1.0.0
4  # repository: https://github.com/my_org/my_repo
5
6  environment:
7    sdk: ^3.9.2
8
9  # Add regular dependencies here.
10 dependencies:
11   path: ^1.9.0
12   http: ^1.6.0
13
14 dev_dependencies:
15   lints: ^6.0.0
16   test: ^1.25.6
17
```

ForecastProvider

```
forecast_provider.dart > ...
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'weather_data.dart';

class ForecastProvider {
  List<WeatherData> hourlyForecast = [];

  Future<bool> fetchHourlyForecast() async {
    hourlyForecast.clear();

    // latitude and longitude of Schloß Johannisburg in Aschaffenburg
    var lat = 49.97606;
    var lon = 9.14163;

    // 'https://api.open-meteo.com/v1/forecast?latitude=49.97606&longitude=9.14163&hourly=temperature_2m,precipitation,cloud_cover&forecast_days=1';
    final url =
      "https://api.open-meteo.com/v1/forecast?latitude=$lat&longitude=$lon&hourly=temperature_2m,precipitation,cloud_cover&forecast_days=1";

    final response = await http.get(Uri.parse(url));
    if (response.statusCode != 200) {
      print(
        "statusCode in response from open-meteo api was ${response.statusCode}",
      );
      return (false);
    }

    final data = jsonDecode(response.body);
```

```
Run | Debug
void main(List<String> arguments) async {
  var forecastProvider = ForecastProvider();

  bool succeeded = await forecastProvider.fetchHourlyForecast();
  if (!succeeded) {
    print("fetchHourlyForecast did not succeed.");
    return;
  }

  for (var e in forecastProvider.hourlyForecast) {
    print("${e.localTime}: ${e.temp}°C, ${e.precipAmount}mm, ${e.cloudCover}%");
  }
}
```



ForecastProvider continued

```
final data = jsonDecode(response.body);

List<String> times = [];
List<double?> temps = [];
List<double?> precipAmounts = [];
List<int?> cloudCovers = [];

times = List<String>.from(data['hourly']['time']);
temps = List<double?>.from(data['hourly']['temperature_2m']);
precipAmounts = List<double?>.from(data['hourly']['precipitation']);
cloudCovers = List<int?>.from(data['hourly']['cloud_cover']);

for (int i = 0; i < times.length; i++) {
  final localTime = DateTime.parse(times[i]);
  var data = WeatherData(localTime);
  data.temp = temps[i];
  data.precipAmount = precipAmounts[i];
  data.cloudCover = cloudCovers[i];
  hourlyForecast.add(data);
}
return true;
}
```

```
{
  "latitude": 49.98,
  "longitude": 9.139999,
  "generationtime_ms": 0.0771284103393555,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 124,
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "°C",
    "precipitation": "mm",
    "cloud_cover": "%"
  },
  "hourly": {
    "time": [
      "2026-03-22T00:00",
      "2026-03-22T01:00",
      "2026-03-22T02:00",
      "2026-03-22T23:00"
    ],
    "temperature_2m": [7.1, 6.6, 6.1, 5.5,
      "precipitation": [0, 0, 0, 0, 0, 0, 0,
      "cloud_cover": [47, 57, 0, 0, 0, 0, 0,
```

```
weather_data.dart X
lib > weather_data.dart > WeatherData
1 class WeatherData {
2   DateTime localTime;
3   double? temp;
4   int? cloudCover;
5   double? precipAmount;
6
7   WeatherData(this.localTime);
8 }
```

Values in these lists can be null, see

https://api.open-meteo.com/v1/forecast?latitude=47.37&longitude=8.55&hourly=temperature_2m&models=meteoswiss_icon_ch1&forecast_days=3

First results

```
"hourly": {  
  "time": [  
    "2026-03-22T00:00",  
    "2026-03-22T01:00",  
    "2026-03-22T02:00",  
    "2026-03-22T03:00",  
    "2026-03-22T04:00",  
    "2026-03-22T05:00",  
    "2026-03-22T06:00",  
    "2026-03-22T07:00",  
    "2026-03-22T08:00",  
    "2026-03-22T09:00",  
    "2026-03-22T10:00",  
    "2026-03-22T11:00",  
    "2026-03-22T12:00",  
    "2026-03-22T13:00",  
    "2026-03-22T14:00",  
    "2026-03-22T15:00",  
    "2026-03-22T16:00",  
    "2026-03-22T17:00",  
    "2026-03-22T18:00",  
    "2026-03-22T19:00",  
    "2026-03-22T20:00",  
    "2026-03-22T21:00",  
    "2026-03-22T22:00",  
    "2026-03-22T23:00"  
  ],  
  "temperature_2m": [7.1, 6.6, 6.1, 5.5, 5, 4.7, 4.8, 5.9, 7.6, 10.1, 12.2, 13.5, 14.1, 15.4, 15.3, 15.2, 15, 14.6, 12.9, 11.5, 10.2, 10.1, 9.4, 8.7],  
  "precipitation": [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],  
  "cloud_cover": [47, 57, 0, 0, 0, 0, 0, 0, 0, 0, 0, 5, 43, 96, 64, 37, 40, 36, 60, 0, 59, 67, 64, 100]  
}
```

```
Connected to the VM Service.  
2026-03-22 00:00:00.000: 7.1°C, 0.0mm, 47%  
2026-03-22 01:00:00.000: 6.6°C, 0.0mm, 57%  
2026-03-22 02:00:00.000: 6.1°C, 0.0mm, 0%  
2026-03-22 03:00:00.000: 5.5°C, 0.0mm, 0%  
2026-03-22 04:00:00.000: 5.0°C, 0.0mm, 0%  
2026-03-22 05:00:00.000: 4.7°C, 0.0mm, 0%  
2026-03-22 06:00:00.000: 4.8°C, 0.0mm, 0%  
2026-03-22 07:00:00.000: 5.9°C, 0.0mm, 0%  
2026-03-22 08:00:00.000: 7.6°C, 0.0mm, 0%  
2026-03-22 09:00:00.000: 10.1°C, 0.0mm, 0%  
2026-03-22 10:00:00.000: 12.2°C, 0.0mm, 0%  
2026-03-22 11:00:00.000: 13.5°C, 0.0mm, 5%  
2026-03-22 12:00:00.000: 14.1°C, 0.0mm, 43%  
2026-03-22 13:00:00.000: 15.4°C, 0.0mm, 96%  
2026-03-22 14:00:00.000: 15.3°C, 0.0mm, 64%  
2026-03-22 15:00:00.000: 15.2°C, 0.0mm, 37%  
2026-03-22 16:00:00.000: 15.0°C, 0.0mm, 40%  
2026-03-22 17:00:00.000: 14.6°C, 0.0mm, 36%  
2026-03-22 18:00:00.000: 12.9°C, 0.0mm, 60%  
2026-03-22 19:00:00.000: 11.5°C, 0.0mm, 0%  
2026-03-22 20:00:00.000: 10.2°C, 0.0mm, 59%  
2026-03-22 21:00:00.000: 10.1°C, 0.0mm, 67%  
2026-03-22 22:00:00.000: 9.4°C, 0.0mm, 64%  
2026-03-22 23:00:00.000: 8.7°C, 0.0mm, 100%
```





Same for Sydney

```
// latitude and longitude of Schloß Johannisburg in Aschaffenburg
// var lat = 49.97606;
// var lon = 9.14163;

// latitude and longitude for Sydney
var lat = -33.86785;
var lon = 151.20732;
```

```
Connected to the VM Service.
2026-03-22 00:00:00.000: 23.4°C, 0.1mm, 81%
2026-03-22 01:00:00.000: 24.2°C, 0.1mm, 67%
2026-03-22 02:00:00.000: 24.9°C, 0.1mm, 64%
2026-03-22 03:00:00.000: 25.5°C, 0.1mm, 72%
2026-03-22 04:00:00.000: 25.8°C, 0.1mm, 72%
2026-03-22 05:00:00.000: 25.8°C, 0.0mm, 48%
2026-03-22 06:00:00.000: 25.7°C, 0.0mm, 38%
2026-03-22 07:00:00.000: 24.5°C, 0.0mm, 40%
2026-03-22 08:00:00.000: 23.2°C, 0.0mm, 38%
2026-03-22 09:00:00.000: 22.0°C, 0.0mm, 39%
2026-03-22 10:00:00.000: 21.3°C, 0.0mm, 61%
2026-03-22 11:00:00.000: 20.7°C, 0.0mm, 66%
2026-03-22 12:00:00.000: 19.9°C, 0.0mm, 52%
2026-03-22 13:00:00.000: 19.4°C, 0.0mm, 54%
2026-03-22 14:00:00.000: 19.0°C, 0.0mm, 59%
2026-03-22 15:00:00.000: 18.9°C, 0.0mm, 55%
2026-03-22 16:00:00.000: 18.8°C, 0.0mm, 64%
2026-03-22 17:00:00.000: 18.4°C, 0.0mm, 62%
2026-03-22 18:00:00.000: 18.3°C, 0.0mm, 64%
2026-03-22 19:00:00.000: 18.1°C, 0.0mm, 55%
2026-03-22 20:00:00.000: 17.9°C, 0.0mm, 53%
2026-03-22 21:00:00.000: 18.4°C, 0.0mm, 74%
2026-03-22 22:00:00.000: 20.2°C, 0.1mm, 78%
2026-03-22 23:00:00.000: 21.7°C, 0.2mm, 83%
```

Temperature
peak during
the night ?



Timestamps in the response

Location and Time

Location: ☒ Coordinates ☐ List ☐ Bounding box

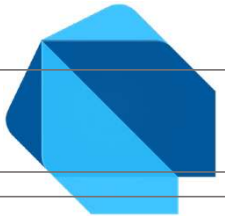
Latitude: 52.52 Longitude: 13.41 Timezone: Not set (GMT+0)

Time: ☒ Forecast Length ☐ Time Interval

Forecast days: 1 day Past days: 0 days (default)

- Not set (GMT+0) ✓
- Automatically detect time zone
- GMT+0
- America/Anchorage
- America/Los Angeles

Timestamps in the result are in UTC (Coordinated Universal Time).



On the left:

https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&hourly=temperature_2m

On the right:

https://api.open-meteo.com/v1/forecast?latitude=52.52&longitude=13.41&hourly=temperature_2m&timezone=auto

```
{
  "latitude": 52.52,
  "longitude": 13.419998,
  "generationtime_ms": 0.0767707824707031,
  "utc_offset_seconds": 0,
  "timezone": "GMT",
  "timezone_abbreviation": "GMT",
  "elevation": 38,
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "°C"
  },
  "hourly": {
    "time": [
      "2026-05-11T00:00",
      "2026-05-11T01:00",
      "2026-05-11T02:00",
      "2026-05-11T03:00",
      "2026-05-11T04:00",
      "2026-05-11T05:00",
      "2026-05-11T06:00",
      "2026-05-11T07:00",
      "2026-05-11T08:00",
      "2026-05-11T09:00",
      "2026-05-11T10:00",
      "2026-05-11T11:00",
      "2026-05-11T12:00",
    ],
    "temperature_2m": [13.5, 12.9, 12.4, 12.6, 12.5, 12.5, 13.1,
13.9, 14.9, 15.1, 16, 16.6, 16.5, 14.3, 13.3, 14.2, 12.9, 11.4,
```

```
{
  "latitude": 52.52,
  "longitude": 13.419998,
  "generationtime_ms": 0.0550746917724609,
  "utc_offset_seconds": 7200,
  "timezone": "Europe/Berlin",
  "timezone_abbreviation": "GMT+2",
  "elevation": 38,
  "hourly_units": {
    "time": "iso8601",
    "temperature_2m": "°C"
  },
  "hourly": {
    "time": [
      "2026-05-11T00:00",
      "2026-05-11T01:00",
      "2026-05-11T02:00",
      "2026-05-11T03:00",
      "2026-05-11T04:00",
      "2026-05-11T05:00",
      "2026-05-11T06:00",
      "2026-05-11T07:00",
      "2026-05-11T08:00",
      "2026-05-11T09:00",
      "2026-05-11T10:00",
      "2026-05-11T11:00",
      "2026-05-11T12:00",
    ],
    "temperature_2m": [14.4, 13.7, 13.5, 12.9, 12.4, 12.6, 12.5,
12.5, 13.1, 13.9, 14.9, 15.1, 16, 16.6, 16.5, 14.3, 13.3, 14.2,
```





Convert UTC to Sydney local time

<https://chatgpt.com/share/69bc5907-059c-8003-b4c4-815c9bccfa24>

ChatGPT ▾

In Dart, when I have a `DateTime` in UTC, how to find out this time in Sydney

In Dart, you can convert a UTC `DateTime` to Sydney time by using the `toLocal()` method **only if your system timezone is already set to Sydney**. But usually, that's not reliable—so the proper way is to use a `timezone` package.

✅ **Best approach: use the `timezone` package**

1. Add dependency:

```
YAML

dependencies:
  timezone: ^0.9.0
```



🔥 Summary

- Use `timezone` package → correct & DST-safe
- Avoid manual offsets
- `toLocal()` only works if system timezone matches Sydney

2. Initialize and convert:

```
dart

import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tz;

void main() {
  // Initialize timezone database
  tz.initializeTimeZones();

  // Your UTC time
  DateTime utcTime = DateTime.utc(2026, 3, 19, 12, 0);

  // Get Sydney Location
  final sydney = tz.getLocation('Australia/Sydney');

  // Convert
  final sydneyTime = tz.TZDateTime.from(utcTime, sydney);

  print(sydneyTime);
}
```

💡 Why this is needed

Sydney uses daylight saving time (AEDT/AEST), so the offset is not always the same:

- UTC+10 (standard time)
- UTC+11 (daylight saving)

The `timezone` package handles this automatically.



Get timezone name from lat/lon

<https://gemini.google.com/share/d75c20b14e7f>

In Dart if I have latitude and longitude of a location, how to get its timezone name like 'Australia/Sydney' ?



In Dart, getting a timezone name (like 'Australia/Sydney') from coordinates is a bit tricky because the language doesn't have a built-in database of geographic boundaries. You generally have two ways to handle this: using an **online API** (easiest/most accurate) or an **offline package** (better for privacy/no internet).

Option 1: Using an Offline Package (Recommended)

The most popular way to do this without hitting a server is the `lat_lng_to_timezone` package. It uses a local database to look up which timezone polygon a coordinate falls into.

1. **Add the dependency** to your `pubspec.yaml`:

YAML

```
dependencies:  
  lat_lng_to_timezone: ^1.0.2
```

2. Use it in your code:

Dart

```
import 'package:lat_lng_to_timezone/lat_lng_to_timezone.dart' as tzlookup;  
  
void main() {  
  double lat = -33.8688;  
  double lon = 151.2093;  
  
  String timezone = tzlookup.getTimezone(lat, lon);  
  print('Timezone: $timezone'); // Output: Australia/Sydney  
}
```



Gemini's answer was misleading !

<https://gemini.google.com/share/d75c20b14e7f>

getTimeZone does not exist in lat_lng_to_timezone. What about latLngToTimezoneString



You've got it. I clearly need to recalibrate my internal map of this specific package. You are absolutely correct—the function name in `lat_lng_to_timezone` (v0.2.0) is indeed `latLngToTimezoneString`.

Here is the **exact** implementation that will compile and run with that package:

Verified Implementation (`lat_lng_to_timezone: ^0.2.0`)

Dart

```
import 'package:lat_lng_to_timezone/lat_lng_to_timezone.dart' as tz;

void main() {
  double lat = -33.8688;
  double lon = 151.2093;

  // This is the correct function name for version 0.2.0
  String timezone = tz.latLngToTimezoneString(lat, lon);

  print('Timezone: $timezone'); // Australia/Sydney
}
```

BTW: ChatGPT's answer wasn't correct too, see:
<https://chatgpt.com/share/697a8838-b92c-8003-9d94-aecd77210337>



Converting UTC to local times

```
forecast_provider.dart > ForecastProvider > _initialize
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'package:lat_lng_to_timezone/lat_lng_to_timezone.dart';
import 'package:timezone/data/latest.dart' as tz;
import 'package:timezone/timezone.dart' as tz;
import 'weather_data.dart';

class ForecastProvider {
  List<WeatherData> hourlyForecast = [];
  bool _initialized = false;

  void _initialize() {
    tz.initializeTimeZones();
  }

  Future<bool> fetchHourlyForecast() async {
    if (!_initialized) {
      _initialize();
      _initialized = true;
    }

    hourlyForecast.clear();
  }
}
```

```
times = List<String>.from(data['hourly']['time']);
temps = List<double?>.from(data['hourly']['temperature_2m']);
precipAmounts = List<double?>.from(data['hourly']['precipitation']);
cloudCovers = List<int?>.from(data['hourly']['cloud_cover']);

var timezoneName = latLngToTimezoneString(lat, lon);
final tzLocation = tz.getLocation(timezoneName);

for (int i = 0; i < times.length; i++) {
  final timeInUtc = DateTime.parse(times[i]);
  final localTime = tz.TZDateTime.from(timeInUtc, tzLocation);
  var data = WeatherData(localTime);
  data.temp = temps[i];
  data.precipAmount = precipAmounts[i];
  data.cloudCover = cloudCovers[i];
  hourlyForecast.add(data);
}
```



Results with Sydney local times

Before with
UTC times:

```
Connected to the VM Service.
2026-03-22 00:00:00.000: 23.4°C, 0.1mm, 81%
2026-03-22 01:00:00.000: 24.2°C, 0.1mm, 67%
2026-03-22 02:00:00.000: 24.9°C, 0.1mm, 64%
2026-03-22 03:00:00.000: 25.5°C, 0.1mm, 72%
2026-03-22 04:00:00.000: 25.8°C, 0.1mm, 72%
2026-03-22 05:00:00.000: 25.8°C, 0.0mm, 48%
2026-03-22 06:00:00.000: 25.7°C, 0.0mm, 38%
2026-03-22 07:00:00.000: 24.5°C, 0.0mm, 40%
2026-03-22 08:00:00.000: 23.2°C, 0.0mm, 38%
2026-03-22 09:00:00.000: 22.0°C, 0.0mm, 39%
2026-03-22 10:00:00.000: 21.3°C, 0.0mm, 61%
2026-03-22 11:00:00.000: 20.7°C, 0.0mm, 66%
2026-03-22 12:00:00.000: 19.9°C, 0.0mm, 52%
2026-03-22 13:00:00.000: 19.4°C, 0.0mm, 54%
2026-03-22 14:00:00.000: 19.0°C, 0.0mm, 59%
2026-03-22 15:00:00.000: 18.9°C, 0.0mm, 55%
2026-03-22 16:00:00.000: 18.8°C, 0.0mm, 64%
2026-03-22 17:00:00.000: 18.4°C, 0.0mm, 62%
2026-03-22 18:00:00.000: 18.3°C, 0.0mm, 64%
2026-03-22 19:00:00.000: 18.1°C, 0.0mm, 55%
2026-03-22 20:00:00.000: 17.9°C, 0.0mm, 53%
2026-03-22 21:00:00.000: 18.4°C, 0.0mm, 74%
2026-03-22 22:00:00.000: 20.2°C, 0.1mm, 78%
2026-03-22 23:00:00.000: 21.7°C, 0.2mm, 83%
```

Now with
local times:

Should start with 11:00 ?!

```
Connected to the VM Service.
2026-03-22 10:00:00.000+1100: 23.4°C, 0.1mm, 81%
2026-03-22 11:00:00.000+1100: 24.2°C, 0.1mm, 67%
2026-03-22 12:00:00.000+1100: 24.9°C, 0.1mm, 64%
2026-03-22 13:00:00.000+1100: 25.5°C, 0.1mm, 72%
2026-03-22 14:00:00.000+1100: 25.8°C, 0.1mm, 72%
2026-03-22 15:00:00.000+1100: 25.8°C, 0.0mm, 48%
2026-03-22 16:00:00.000+1100: 25.7°C, 0.0mm, 38%
2026-03-22 17:00:00.000+1100: 24.5°C, 0.0mm, 40%
2026-03-22 18:00:00.000+1100: 23.2°C, 0.0mm, 38%
2026-03-22 19:00:00.000+1100: 22.0°C, 0.0mm, 39%
2026-03-22 20:00:00.000+1100: 21.3°C, 0.0mm, 61%
2026-03-22 21:00:00.000+1100: 20.7°C, 0.0mm, 66%
2026-03-22 22:00:00.000+1100: 19.9°C, 0.0mm, 52%
2026-03-22 23:00:00.000+1100: 19.4°C, 0.0mm, 54%
2026-03-23 00:00:00.000+1100: 19.0°C, 0.0mm, 59%
2026-03-23 01:00:00.000+1100: 18.9°C, 0.0mm, 55%
2026-03-23 02:00:00.000+1100: 18.8°C, 0.0mm, 64%
2026-03-23 03:00:00.000+1100: 18.4°C, 0.0mm, 62%
2026-03-23 04:00:00.000+1100: 18.3°C, 0.0mm, 64%
2026-03-23 05:00:00.000+1100: 18.1°C, 0.0mm, 55%
2026-03-23 06:00:00.000+1100: 17.9°C, 0.0mm, 53%
2026-03-23 07:00:00.000+1100: 18.4°C, 0.0mm, 74%
2026-03-23 08:00:00.000+1100: 20.2°C, 0.1mm, 78%
2026-03-23 09:00:00.000+1100: 21.7°C, 0.2mm, 83%
```



Time zones in Dart

ChatGPT ▾

does dart support timezones

Short answer: Yes, but only partially.

Dart does not natively support full IANA time zones (like "Australia/Sydney" or "America/New_York").

The built-in `DateTime` can only handle:

What Dart *does* support natively:

- UTC time (`DateTime.utc(...)`)
- Local device time zone (`DateTime.now()`)
- Fixed UTC offsets (e.g., `DateTime.now().toUtc().add(Duration(hours: 10))`)

What Dart *does NOT* support natively:

- ❌ Named time zones like "Europe/Paris"
- ❌ Automatic daylight saving transitions
- ❌ Time zone database (IANA tzdb)

Local device time and time in UTC



```
datetime_tests.dart > ...  
Run | Debug  
void main() {  
  var now = DateTime.now();  
  print("now: $now");  
  var nowInUtc = now.toUtc();  
  print("nowInUtc: $nowInUtc");  
  var backToLocal = nowInUtc.toLocal();  
  print("backToLocal: $backToLocal");  
  
  assert(now.isUtc == false);  
  assert(nowInUtc.isUtc == true);  
  assert(backToLocal.isUtc == false);  
}
```

```
now: 2026-03-22 22:55:50.295847  
nowInUtc: 2026-03-22 21:55:50.295847Z  
backToLocal: 2026-03-22 22:55:50.295847
```

```
var parsed = DateTime.parse("2026-03-22T00:00");  
print("parsed: $parsed");  
assert(parsed.isUtc == false);  
  
var parsedZ = DateTime.parse("2026-03-22T00:00Z");  
print("parsedZ: $parsedZ");  
assert(parsedZ.isUtc == true);  
}
```

```
parsed: 2026-03-22 00:00:00.000  
parsedZ: 2026-03-22 00:00:00.000Z
```


Correct parsing of times from OpenMeteo



```
var timeZoneName = latLngToTimezoneString(lat, lon);
final tzLocation = tz.getLocation(timeZoneName);

for (int i = 0; i < times.length; i++) {
    final timeInUtc = DateTime.parse("${times[i]}Z");
    final localTime = tz.TZDateTime.from(timeInUtc, tzLocation);
    var data = WeatherData(localTime);
    data.temp = temps[i];
}
```

```
Connected to the VM Service.
2026-03-22 11:00:00.000+1100: 23.4°C, 0.1mm, 81%
2026-03-22 12:00:00.000+1100: 24.2°C, 0.1mm, 67%
2026-03-22 13:00:00.000+1100: 24.9°C, 0.1mm, 64%
2026-03-22 14:00:00.000+1100: 25.5°C, 0.1mm, 72%
2026-03-22 15:00:00.000+1100: 25.8°C, 0.1mm, 72%
2026-03-22 16:00:00.000+1100: 25.8°C, 0.0mm, 48%
2026-03-22 17:00:00.000+1100: 25.7°C, 0.0mm, 38%
2026-03-22 18:00:00.000+1100: 24.5°C, 0.0mm, 40%
2026-03-22 19:00:00.000+1100: 23.2°C, 0.0mm, 38%
2026-03-22 20:00:00.000+1100: 22.0°C, 0.0mm, 39%
2026-03-22 21:00:00.000+1100: 21.3°C, 0.0mm, 61%
2026-03-22 22:00:00.000+1100: 20.7°C, 0.0mm, 66%
2026-03-22 23:00:00.000+1100: 19.9°C, 0.0mm, 52%
2026-03-23 00:00:00.000+1100: 19.4°C, 0.0mm, 54%
2026-03-23 01:00:00.000+1100: 19.0°C, 0.0mm, 59%
2026-03-23 02:00:00.000+1100: 18.9°C, 0.0mm, 55%
2026-03-23 03:00:00.000+1100: 18.8°C, 0.0mm, 64%
2026-03-23 04:00:00.000+1100: 18.4°C, 0.0mm, 62%
2026-03-23 05:00:00.000+1100: 18.9°C, 0.0mm, 84%
2026-03-23 06:00:00.000+1100: 18.9°C, 0.1mm, 80%
2026-03-23 07:00:00.000+1100: 19.0°C, 0.2mm, 84%
2026-03-23 08:00:00.000+1100: 19.3°C, 0.1mm, 88%
2026-03-23 09:00:00.000+1100: 20.4°C, 0.1mm, 90%
2026-03-23 10:00:00.000+1100: 21.4°C, 0.2mm, 86%
```


Goal: only show future values in the output



```
var timezoneName = latLngToTimezoneString(lat, lon);
final tzLocation = tz.getLocation(timezoneName);
final nowInUtc = DateTime.now().toUtc();

for (int i = 0; i < times.length; i++) {
    final timeInUtc = DateTime.parse("${times[i]}Z");
    if (timeInUtc.isAfter(nowInUtc)) {
        final localTime = tz.TZDateTime.from(timeInUtc, tzLocation);
        var data = WeatherData(localTime);
        data.temp = temps[i];
        data.precipAmount = precipAmounts[i];
        data.cloudCover = cloudCovers[i];
        hourlyForecast.add(data);
    }
}
```

```
Connected to the VM Service.
2026-03-23 21:00:00.000+0100: 9.5°C, 0.0mm, 1%
2026-03-23 22:00:00.000+0100: 8.6°C, 0.0mm, 56%
2026-03-23 23:00:00.000+0100: 7.6°C, 0.0mm, 0%
2026-03-24 00:00:00.000+0100: 6.7°C, 0.0mm, 17%
```

Values for Sydney



Aktuelle Uhrzeit / Ortszeit

Aschaffenburg
Europe/Berlin

Mo., 23. Mär. 2026

20:32:12

[HTML](#)

Aktuelle Uhrzeit / Ortszeit

Sydney
Australia/Sydney

Di., 24. Mär. 2026

06:32:12

[HTML](#)

```
Connected to the VM Service.
2026-03-24 07:00:00.000+1100: 18.0°C, 0.0mm, 61%
2026-03-24 08:00:00.000+1100: 18.1°C, 0.0mm, 69%
2026-03-24 09:00:00.000+1100: 20.6°C, 0.0mm, 46%
2026-03-24 10:00:00.000+1100: 22.5°C, 0.0mm, 55%
```

Android Emulator - Medium_Phone_API_36.1:5554

20:34

Open-Meteo UI

Weather in: Sydney

Forecast for Tue 24-Mar 07:00 (local time)

temp: 18.0°C
clouds: 61%
wind: 3km/h A

↑ 07:01 ↓ 19:01
↑ 12:42 ↓ 22:21

Tue 07:00	☁	18.0°C	☀ 61%	💧 0%
Tue 08:00	☁	18.1°C	☀ 69%	💧 0%
Tue 09:00	☁	20.6°C	☀ 46%	💧 0%
Tue 10:00	☁	22.5°C	☀ 55%	💧 0%
Tue 11:00	☁	24.0°C	☀ 53%	💧 3%
Tue 12:00	☁	25.3°C	☀ 68%	💧 3%
Tue 13:00	☁	26.5°C	☀ 68%	💧 3%



What you should have learned in this lesson

- Know 2 ways how to create a “professional” Dart console app
- Understand the role of the sub-folders bin, lib and test of such an app
- Read weather forecast via http from Open Meteo
- Know how time-zones are managed in Dart
- Find out current time in Sydney in Dart

Appendix 1: Some tests with Daylight Saving Time (DST)



```
// DST winter -> summer :  
  
var winSomInUTC = DateTime.parse("2026-03-29T00:00Z");  
for (int i = 0; i < 6; i++) {  
    var dt = winSomInUTC.add(Duration(hours: i));  
    print("DateTime in UTC: $dt");  
}
```

```
var winSomLocal = winSomInUTC.toLocal();  
for (int i = 0; i < 6; i++) {  
    var dt = winSomLocal.add(Duration(hours: i));  
    print("DateTime local: $dt");  
}
```

```
tz.initializeTimeZones();  
var timezoneName = "Europe/Berlin";  
final tzLocation = tz.getLocation(timezoneName);  
var winSomTZ = tz.TZDateTime.from(winSomInUTC, tzLocation);  
for (int i = 0; i < 6; i++) {  
    var dt = winSomTZ.add(Duration(hours: i));  
    print("with TZDateTime: $dt");  
}
```

```
DateTime in UTC: 2026-03-29 00:00:00.000Z  
DateTime in UTC: 2026-03-29 01:00:00.000Z  
DateTime in UTC: 2026-03-29 02:00:00.000Z  
DateTime in UTC: 2026-03-29 03:00:00.000Z  
DateTime in UTC: 2026-03-29 04:00:00.000Z  
DateTime in UTC: 2026-03-29 05:00:00.000Z
```

```
DateTime local: 2026-03-29 01:00:00.000  
DateTime local: 2026-03-29 03:00:00.000  
DateTime local: 2026-03-29 04:00:00.000  
DateTime local: 2026-03-29 05:00:00.000  
DateTime local: 2026-03-29 06:00:00.000  
DateTime local: 2026-03-29 07:00:00.000
```

```
with TZDateTime: 2026-03-29 01:00:00.000+0100  
with TZDateTime: 2026-03-29 03:00:00.000+0200  
with TZDateTime: 2026-03-29 04:00:00.000+0200  
with TZDateTime: 2026-03-29 05:00:00.000+0200  
with TZDateTime: 2026-03-29 06:00:00.000+0200  
with TZDateTime: 2026-03-29 07:00:00.000+0200
```


Appendix 2: DateTime and the “Unix Epoch”



```
var now = DateTime.now();
print("now: $now");
var nowInUtc = now.toUtc();
print("nowInUtc: $nowInUtc");

print("now.microsecondsSinceEpoch: ${now.microsecondsSinceEpoch}");
print("nowInUtc.microsecondsSinceEpoch: ${nowInUtc.microsecondsSinceEpoch}");
var dtEpoch = DateTime.fromMicrosecondsSinceEpoch(0, isUtc: true);
print("dtEpoch: $dtEpoch");
var dDay = DateTime.utc(1944, 6, 6);
print("dday.microsecondsSinceEpoch: ${dDay.microsecondsSinceEpoch}");
```

```
now: 2026-03-23 18:47:38.622078
nowInUtc: 2026-03-23 17:47:38.622078Z
now.microsecondsSinceEpoch: 1774288058622078
nowInUtc.microsecondsSinceEpoch: 1774288058622078
dtEpoch: 1970-01-01 00:00:00.000Z
dday.microsecondsSinceEpoch: -806976000000000
```

```
C: > flutter > sdk > flutter > bin > cache > dart-sdk > lib > core > date_time.dart > DateTime
136 class DateTime implements Comparable<DateTime> {
802   /// The number of microseconds since
803   /// the "Unix epoch" 1970-01-01T00:00:00Z (UTC).
804   ///
805   /// This value is independent of the time zone.
806   ///
807   /// This value is at most
808   /// 8,640,000,000,000,000,000us (100,000,000 days) from the Unix epoch.
809   /// In other words: `microsecondsSinceEpoch.abs() <= 8640000000000000000`.
810   ///
811   /// Note that this value does not always fit into 53 bits (the size of a IEEE
812   /// double). On the web JavaScript platforms, there may be a rounding error
813   /// for DateTime values sufficiently far from the epoch. The year range close
814   /// to the epoch to avoid rounding is approximately 1685..2254.
815   external int get microsecondsSinceEpoch;
```

```
// max micro seconds:      8,640,000,000,000,000,000
// 24*3600*1000*1000 (1 day): 86,400,000,000
// max int native (64bit):  9,223,372,036,854,775,807 = 2^63 - 1
```



Max int on different platforms

What is the maximum int value in Flutter. Does it depend on the platform ?

<https://gemini.google.com/share/555dfea03b2b>



Google Search

The short answer is **yes**, the maximum value of an `int` in Flutter (Dart) depends entirely on the platform where your code is running.

Because Dart can compile to both native machine code (iOS/Android/Desktop) and JavaScript (Web), it has to navigate two different ways of handling numbers.

1. Native Platforms (iOS, Android, Windows, macOS, Linux)

On native platforms, Dart uses **64-bit signed two's complement integers**. This is the standard behavior for most modern programming languages.

- **Max Value:** $2^{63} - 1$ (or **9,223,372,036,854,775,807**)
- **Min Value:** -2^{63} (or **-9,223,372,036,854,775,808**)

2. Web Platforms (Chrome, Safari, etc.)

When Dart is compiled to JavaScript, `int` values are represented as **JavaScript numbers**.

In JS, all numbers are actually 64-bit floating-point values (doubles). This means they can only represent integers exactly up to a certain point.

- **Max Safe Integer:** $2^{53} - 1$ (or **9,007,199,254,740,991**)
- **Behavior:** Beyond this limit, you lose "precision." For example, on the web, `9007199254740992 + 1` might still result in `9007199254740992` because the bits required to store that "1" are no longer available in the floating-point mantissa.



Some tests on max. MicroSecondsSinceEpoch

```
// max micro seconds:      8,640,000,000,000,000,000
// 24*3600*1000*1000 (1 day): 86,400,000,000
// max int native (64bit):   9,223,372,036,854,775,807 = 2^63 - 1
```

```
var microsec = 8640000000000000000 - 86400000000;
var test = DateTime.fromMicrosecondsSinceEpoch(microsec, isUtc: true);
print(test); // 275760-09-12 00:00:00.000Z

test = test.add(Duration(days: 1));
print(test); // 275760-09-13 00:00:00.000Z

//test = test.add(Duration(days: 1)); // RangeError
```

```
test = DateTime(275760);
print(test); // 275760-01-01 00:00:00.000

//test = DateTime(275761); // ArgumentError
//test = DateTime(312000); // ArgumentError

// Strange: following line does not throw Exception
test = DateTime(313000);
print(test); // -271555-12-13 15:58:10.448384
```

```
(new) DateTime(
  int year, [
    int month = 1,
    int day = 1,
    int hour = 0,
    int minute = 0,
    int second = 0,
    int millisecond = 0,
    int microsecond = 0,
  ])
```

Declared in `DateTime` in `dart:core`.

Constructs a `DateTime` instance specified in the local time zone.